ESCUELA ESPECIALIZADA EN INGENIERÍA ITCA-FEPADE, MEGATEC, LA UNIÓN

ESCUELA DE INGENIERÍA DE COMPUTACIÓN

TÉCNICO EN INGENIERÍA DE DESARROLLO DE SOFTWARE

APLICACIÓN DE METODOLOGÍAS ÁGILES Y CONTROL DE LA CALIDAD DEL SOFTWARE

SECCIÓN:

DES02-B

INTEGRANTES:

DANIEL ANTONIO SORTO BLANCO

DAVID ALBERTO ROJAS JIMENES

Fecha de entrega: 22 de octubre de 2026

Caso 1: Clínica Salud Integral

Durante varios años administró sus reservaciones mediante llamadas telefónicas, mensajes de WhatsApp y archivos de Excel. Aunque estos mecanismos permitieron mantener sus operaciones, el crecimiento de la demanda hizo frecuentes los registros duplicados, la dificultad para consultar la disponibilidad de los médicos y el tiempo excesivo dedicado a confirmar o modificar citas. Con el propósito de modernizar sus servicios, contrató a una empresa de desarrollo de software para implementar una plataforma web donde los pacientes pudieran gestionar citas sin depender directamente del personal de recepción.

Problemas identificados

Problemática 1:

La recuperación de contraseña se aprobó sin comprobar el envío real del correo. En el caso quedó como aprobado con una captura del mensaje en pantalla "Se ha enviado un correo con instrucciones" varios pacientes indicaron que nunca recibieron el correo.

Problemática 2:

Las reservas simultáneas no se probaron y fallan en producción controlada.

Dos pacientes reservaron prácticamente al mismo tiempo con el mismo médico, fecha y horario. Ambos recibieron confirmación y el conflicto se detectó en la recepción.

Problemática 3

La cancelación de citas y el historial no tienen pruebas ejecutadas. El caso CP-006 aparece como Sin ejecutar y No disponible. Funcionalidades comprometidas en las historias de usuario no cuentan con evidencia de funcionamiento.

Causas por problemas

Problema 1: La prueba verificó el mensaje en pantalla y no la entrega real del correo; faltó un buzón de pruebas y el criterio de aceptación no exigía comprobar la recepción.

Problemas 2: No hay control de concurrencia ni restricción única en la base de datos, y el caso CP-005 quedó sin ejecutar sin que nadie lo exigiera antes del piloto.

Problema 3: El esfuerzo de pruebas se concentró en el flujo principal y las funciones secundarias quedaron sin ejecutar.

Puntos fuertes

- Se aplicó Scrum con Product Owner, Scrum Master y desarrolladores, con sprints de dos semanas y Product Backlog basado en historias de usuario.
- Se definieron criterios de aceptación para cada funcionalidad.
- Se realizó una prueba piloto con pacientes y personal administrativo, que permitió detectar problemas antes del despliegue general.
- Los casos de inicio de sesión válido e inválido se ejecutaron satisfactoriamente con evidencia (CP-001 y CP-002).
- Existe una matriz de pruebas compartida con identificadores, descripción, estado y evidencia, que es una buena base de trazabilidad.
- La dirección actuó con prudencia al suspender temporalmente la implementación y pedir evidencias.

Puntos de mejora

- Verificar el resultado real, no el mensaje. Comprobar la recepción del correo y el cambio efectivo de la contraseña.
- Diseñar pruebas desde los criterios de aceptación, antes o durante el desarrollo y no al final.
- Reforzar el Definition of Done: ninguna historia se cierra sin pruebas ejecutadas y evidencia asociada.
- Separar la demostración de la validación. La demo informa, pero la aceptación debe apoyarse en pruebas verificables.

Solución Propuesta

Prioridad	Acción	Criterio de verificación
Alta	**Corregir la reserva concurrente:** restricción única en base de datos y control de transacción o bloqueo; si el horario ya fue tomado, mostrar un mensaje claro	Prueba automatizada que lanza dos solicitudes simultáneas para el mismo horario: solo una se confirma y la otra es rechazada con mensaje. Cero duplicados en BD
Alta	**Corregir y verificar la recuperación de contraseña:** revisar el servicio de envío, registrar el resultado del envío y manejar los errores	Prueba de integración con buzón de pruebas: el correo llega con enlace válido, el enlace permite cambiar la contraseña y la contraseña anterior deja de funcionar
Media	**Ejecutar y documentar**	Al cancelar, el horario queda libre, el historial refleja el cambio y existe evidencia en la matriz

Caso 2: Tecnomarket

Una tienda de tecnología que creó una plataforma e-commerce (API REST, web y base de datos) para automatizar sus ventas e inventario, pero enfrenta graves fallas porque sus módulos funcionan bien por separado pero fallan al integrarse (como procesar compras exitosas sin descontar existencias). Además, el equipo presenta deficientes prácticas de calidad: solo verifican códigos HTTP de éxito en Postman sin validar la estructura JSON, no realizan pruebas de regresión ni de valores límite y reportan incidencias de forma informal por chat sin registro de severidad o pasos para reproducirlas.

Problemas identificados

Problemática 1

Regla de descuento sin pruebas de límites. La regla promete 10 % de descuento (por ejemplo, 10 USD en una compra de 100 USD). La prueba unitaria solo validó un caso (la compra de 100 USD) y no se incorporaron pruebas con importes inferiores al umbral (99,99 USD), valores próximos al límite (100,00 y 100,01), entradas inválidas (negativos, cero, texto) ni condiciones excepcionales.

Problemática 2

Validación de la API superficial. En Postman se verificó casi exclusivamente el código HTTP de éxito. No se validó la estructura del JSON ni el cumplimiento de las reglas de negocio. Algunos casos se marcaron aprobados, aunque el cuerpo de la respuesta no coincidía con lo esperado.

Problemática 3

Retrabajo y falta de regresión. Los desarrolladores repiten pruebas manuales, ciertas incidencias consideradas corregidas reaparecen tras cambiar otros módulos, y no existe una batería reutilizable que proteja lo ya corregido.

Causas por problemas

Problema 1.
La prueba unitaria solo valió para una compra de 100 USD. no se aplicaron técnicas de diseño de casos como partición de equivalencia y análisis de valores límite (99,99; 100,00; 100,01), ni casos inválidos Cuando la gerencia pidió ajustes a la lógica de descuentos, el desarrollador los verificó a mano en lugar de ejecutar una batería automatizada

Problema 2.
El pedido se guarda y la API responde HTTP 201, pero las existencias no siempre disminuyen el registro del pedido y el descuento de inventario no se ejecutan como una sola operación de modo que si falla o se retrasa la segunda parte, la primera ya quedó guardada. Como cada módulo se probó por separado y funcionó, no hubo pruebas de integración que revisaran el estado final de la base de datos después de una compra

Problema 3.
Los desarrolladores repiten pruebas manuales y ciertas incidencias reaparecen tras cambios en otros módulos. la colección de Postman y las pruebas unitarias no forman una suite de regresión que se ejecute antes de cada entrega, y no hay presupuesto ni tiempo para ampliar el equipo o adquirir herramientas. Faltan semanas para el lanzamiento y esa presión lleva a corregir sin proteger lo ya corregido

Puntos fuertes

- Existe una API REST con endpoints bien definidos (usuarios, login, productos, pedidos) y una base de datos con registros de prueba.
- Se usan pruebas unitarias y una colección de Postman, así que hay una base de automatización.
- El equipo trabaja por sprints y usa un repositorio con control de versiones.
- La regla de descuento se implementó y la prueba inicial fue satisfactoria.
- El acceso con token inválido a ruta protegida responde correctamente HTTP 401, y el registro válido devuelve 201.

Puntos de mejora

- Pruebas de integración entre pedidos e inventario (y base de datos real o de pruebas) con verificación del estado final.
- Pruebas de valores límite y partición de equivalencia para el descuento, con casos parametrizados.
- Pruebas de contrato de la API: código HTTP correcto, esquema JSON y reglas de negocio (por ejemplo, correo duplicado con 409, importes inválidos con 400/422).

Solución Propuesta

Dado que no hay presupuesto para ampliar el equipo ni para nuevas herramientas, se propone usar lo disponible. La solución propuesta consiste en implementar un plan de pruebas de integración End-to-End en Postman que verifique que las compras descuenten correctamente el inventario en la base de datos y validen la estructura JSON de las respuestas. También se deben agregar pruebas unitarias con valores límite para la regla de descuentos, automatizar las pruebas de regresión en el repositorio de código y corregir los códigos de respuesta de la API

Caso 3: EduGestion

El Instituto de Formación Tecnológica del Oriente utiliza EduGestión para registrar estudiantes, controlar asistencia, ingresar calificaciones y generar reportes académicos. La plataforma centralizó la información y reemplazó los registros manuales; tras resultados positivos con un grupo reducido, la dirección decidió extenderla a más carreras y grupos. El crecimiento trajo mayores exigencias: una cobertura global de pruebas del 80 % que oculta solo 45 % en los casos críticos, duplicación y complejidad de código, tiempos de respuesta superiores al objetivo, interrupciones del servicio y errores de usuario.

Problemas identificados

Problema 1:

La cobertura global alta oculta una cobertura crítica baja. La cobertura de líneas es de 80 %, pero los casos críticos (calificaciones, actualización de datos académicos, autenticación y generación de información administrativa) solo alcanzan 45 %. Las pruebas se concentran en componentes sencillos, mientras que las funciones esenciales cuentan con pocos escenarios de validación.

Problema 2:

Deuda técnica en la estructura del código. Hay 6 % de duplicación y una complejidad ciclomática promedio de 8,2. Parte de la lógica de validación de datos de estudiantes aparece repetida en varios componentes, y ciertos métodos contienen múltiples condiciones y rutas lógicas. Esto dificulta comprender y escribir pruebas representativas, y modificar una regla obliga a intervenir en varios puntos

Problema 3:

Rendimiento por debajo del objetivo. Algunas consultas tienen un tiempo de respuesta de 3,5 segundos frente a un objetivo máximo de 2 segundos. Con jornadas de pocos usuarios funciona bien, pero con carga alta (publicación de notas, consultas de calificaciones) el sistema se vuelve lento. Un grupo de docentes reporta tiempos de 3,5 segundos, o más, ante condiciones de carga previstas.

Causas por problemas

Problemática 1:

Causa directa*:* el indicador de 80 % de líneas se interpreta como señal de calidad, aunque solo el 45 % de los casos críticos está cubierto.

Causa de fondo*:* la cobertura se mide de forma agregada y no por criticidad, y las pruebas automatizadas se concentraron en componentes sencillos y fáciles de probar. Los procesos de calificaciones, actualización de datos académicos, autenticación y generación de información administrativa tienen pocos escenarios de validación.

Problemática 2:

Causa directa: la duplicación es de 6 % y la complejidad promedio de 8,2 la validación de datos de estudiantes está repetida en varios componentes.

Causa de **fondo:** el crecimiento acelerado de la plataforma, con nuevos requerimientos de coordinadores y docentes, se resolvió reutilizando código por copia en lugar de extraerlo a un módulo común, sin refactorizar.

Problemática 3:

Causa directa*:* hay interrupciones ocasionales, algunas coincidentes con picos de acceso, y tiempos de recuperación elevados en ciertos procesos.

Causa de fondo: el monitoreo es insuficiente y los registros de incidencias no capturan el contexto necesario (hora, carga, proceso afectado, error), por lo que no se puede identificar la causa raíz. Tampoco hay procedimientos de recuperación documentados ni metas de tiempo de recuperación, de modo que la restauración depende de la persona disponible

Puntos fuertes

- La plataforma centraliza la información y los usuarios reconocen su valor frente a los procedimientos manuales anteriores
- El equipo detecta riesgos y propone alternativas (centralizar la lógica común, refactorizar), y hubo una evaluación inicial para comprobar la ampliación.
- La dirección quiere decidir con evidencias y no con percepciones, y está dispuesta a escuchar un plan de mejora priorizado.
- Se cuenta con un repositorio de código y pruebas automatizadas sobre varios componentes.

Puntos de mejora

- Reducir la duplicación y la complejidad centralizando la lógica de validación de estudiantes en un servicio o módulo único, con refactorización guiada por pruebas.
- Mejorar la usabilidad: confirmaciones, validaciones, mensajes claros y flujos más cortos en las tareas que generan errores.
- Mejorar la disponibilidad y la recuperación: monitoreo, registros con contexto, alertas y procedimientos de recuperación documentados.

Solución Propuesta

Prioridad	Acción	Criterio de verificación
1	Pruebas sobre los procesos críticos (calificaciones, actualización de datos académicos, autenticación, reportes administrativos), incluyendo casos negativos y de límites	Cobertura de casos críticos de 45 % a 75 % o más, con revisión de que las pruebas tengan aserciones reales

Prioridad	Acción	Criterio de verificación
2	Centralizar la lógica de validación de datos de estudiantes y eliminar duplicados, apoyándose en las pruebas del punto 1	Duplicación de 6 % a 3 % o menos en los módulos intervenidos.
3	Monitoreo y registro de incidencias (tiempos, errores, causa) y procedimiento de recuperación documentado	Toda interrupción queda registrada con causa. Tiempo de recuperación medido y con meta